

微算機系統實習

MICROPROCESSOR SYSTEMS LAB.

AUTUMN, 2026

Instructor : Yen-Lin Chen(陳彥霖), Ph.D. Professor
Dept. Computer Science and Information Engineering
National Taipei University of Technology

LECTURE 4 - 運用CGI技術進行物聯網遠端操控GPIO- COMMON GATEWAY INTERFACE(CGI)



前情提要－我要自學網頁三寶

- **HTML**
- **CSS**
- **JS**

這堂課不是網頁設計，
提問時請針對「嵌入式系統
開發」提問

不要再問林北要學哪個

問就是歐印



學習本章節的目的

- **跨領域技術整合 (IoT Implementation)**
 - 學習如何將網頁前端 (HTML/Form)、後端伺服器 (Node.js/Express) 與嵌入式系統硬體 (TX2 GPIO) 三者串聯，實現完整的物聯網邊緣運算架構。
- **現代化遠端操控 (Remote Hardware Control)**
 - 捨棄傳統複雜的作業系統底層驅動編寫，掌握如何利用 **HTTP 請求 (GET/POST)** 與 **API 路由**，讓任何具備網頁瀏覽器的裝置（手機、平板、電腦）都能遠端監控與操控硬體腳位。
- **從閘道器到硬體控制 (From Gateway to GPIO)**
 - 理解通用閘道器介面 (CGI) 的運作邏輯，並學會使用現代高效能的 **Node.js 異步事件驅動技術**，取代傳統高耗能的 CGI Process，提升嵌入式設備的資料處理效率。
- **實務除錯與部署能力 (Practical Troubleshooting)**
 - 培養在 Linux 終端機 (Terminal) 環境下進行環境建置 (Node.js/npm 安裝)、網路連線診斷 (nmcli/ping/DNS 檢查) 以及解決軟體套件相依性問題的實戰工程能力。



OUTLINE

- 物聯網基本概念
- CGI簡介
- Nodejs簡介
- Nodejs的用途為何？
- npm簡介與安裝使用
- 簡易web server範例



物聯網基本概念

- 物聯網的概念最早出現在比爾蓋茨1995年「未來之路」一書，在書中，比爾蓋茲已經提及Internet of Things的概念
- 1998年，美國麻省理工大學首先提出了當時被稱作「EPC (Electronic Product Code)系統」的物聯網的構想
- 1999年，美國EPCglobal 的Auto-ID中心首先提出物聯網的概念，稱物聯網主要是建立在物品編碼、RFID技術和網際網路的基礎上，實施智慧化識別與管理



物聯網基本概念

- 2005年，ITU發佈了「ITU網際網路報告2005：物聯網」，綜合二者內容，正式提出物聯網的概念，包括了所有物品的聯網和應用，宣稱小至牙刷、大至洗衣機、汽車等都可以透過網路相連與通信
- 2009年，歐盟執委會發表了「物聯網行動計畫」，提出了包括隱私及個資保護、信任和安全、標準化、研究開發、開放和創新、制度意識、國際對話、污染監測管理及未來發展等14項行動內容
- 2012年，特斯拉推出Model S時奠定了車連網的標準，並引入汽車軟體無線網路更新功能，整合成類似的物聯網技術



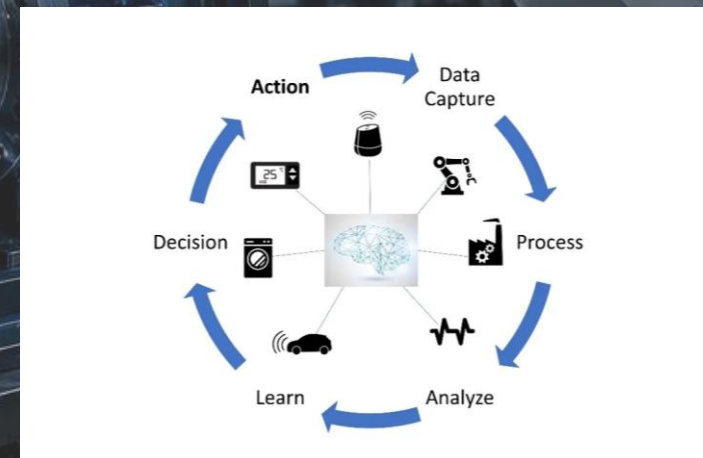
物聯網基本概念

- 2014年，Apple、Google等企業公司陸續發表智慧家庭應用等產品
- 2016年，宏碁、聯發科等國內大廠，成立「亞洲·矽谷物聯網產業大聯盟」，推動「物聯網產業創新研發」及「健全創新創業生態系」
- 2020年代：5G網路的部署加速IoT發展，應用擴展到智慧城市、工業4.0和自動駕駛。AI和邊緣計算整合，提高IoT效率。



物聯網基本概念

- 2024年，IoT技術提升巴黎奧運會的活動管理、裁判和運動員表現。奧運於2024年7月26日至8月11日舉行，使用IoT處理交通、人群管理和安全。
- 2025年，AIoT和邊緣計算大規模整合 AI驅動IoT（AIoT）在工業4.0中普及，邊緣計算實現即時處理；數位孿生在製造和醫療擴展。預計2025年IoT產生80澤位元組資料，等同31億年HD影片播放。





物聯網基本概念

- 物聯網有許多相似的名稱，例如：大陸稱其為物聯網、台灣稱其為智慧聯網、Cisco公司稱其為Internet of Everything (IoE)、美國加州柏克萊大學稱其為Cyber-physical system(CPS)、IBM公司稱其為Smarter Planet (2009年)等
- 物聯網的重點是參與連結之物品會適時產生資訊且都須具備有通訊連結能力，因此感測元件與網路通訊技術自然就成為了物聯網的核心，是一項跨領域之各種技術的整合應用



物聯網基本概念

- 物聯網 (The Internet of Things, IoT) 定義
 - 顧名思義是指將眾多物體(各種裝置和設備)及其所產生的資訊都能經由各式網路相連結，透過現有的網際網路形成巨大的「物物相連的網際網路」，進行資訊收集、發佈、儲存、分析及開發各式可能之應用並進而創造商機
 - 藉由物聯網將使物體具備智慧，使人與人、人和物體、物體和物體之間可以隨時建立相互溝通與對話的環境



物聯網基本概念

- 協定層架構
 - 感知層 (Perception layer)
 - 網路層 (Network layer)
 - 應用層 (Application layer)
- 第一層為「感知層 (Perception layer)」，由各種資訊擷取、識別的感知元件所組成，包括RFID、標籤技術、感測技術、通訊技術以及人工智慧



物聯網基本概念

- 第二層為「網路層 (Network layer)」，即各類無線傳輸技術，包含無線通訊網路、感測網路、EPCglobal網路架構以及新一代網路
- 第三層為「應用層 (Application layer)」，即物聯網的各種應用領域，例如：智慧監測、智慧城市、智慧醫療等
- 物聯網三層架構，如下圖所示



物聯網基本概念

感知層

- RFID
- IP
- Camera
- WSN
- 感測器等等

網路層

- 5G
- Wi-Fi
- Zigbee
- Bluetooth
等等

應用層

- 環境監測
- 智慧居家
- 遠距照護智慧交通
- OPC UA 等等

物聯網3層協定架構



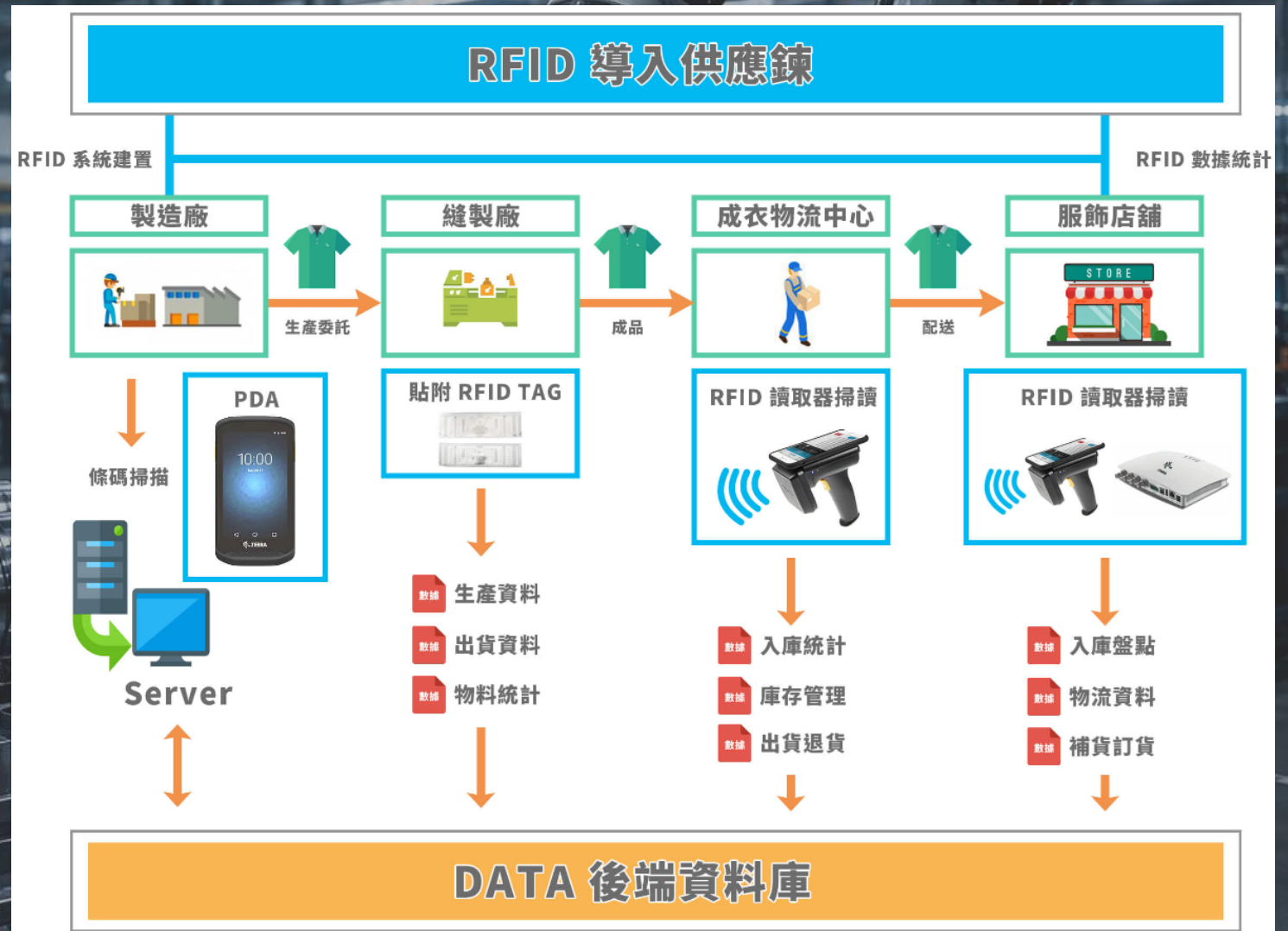
實際案例 - 服飾管理系統

精簡管理作業：

- 顧客結帳
- 盤點存貨
- 更新庫存叫料
- 送貨處理
- 大數據會員資料
- 防止商品被偷竊

效益：

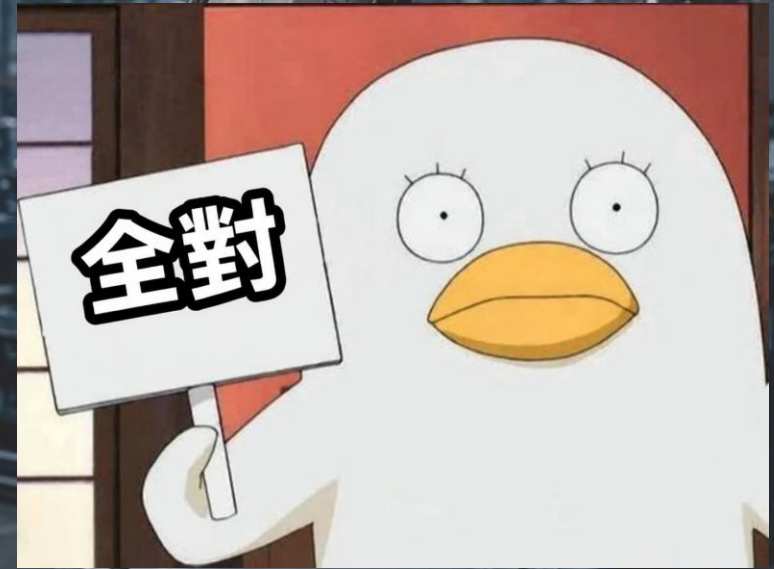
- 快速更新存貨
- 縮短盤點周期
- 控管服飾流向





物聯網基本概念

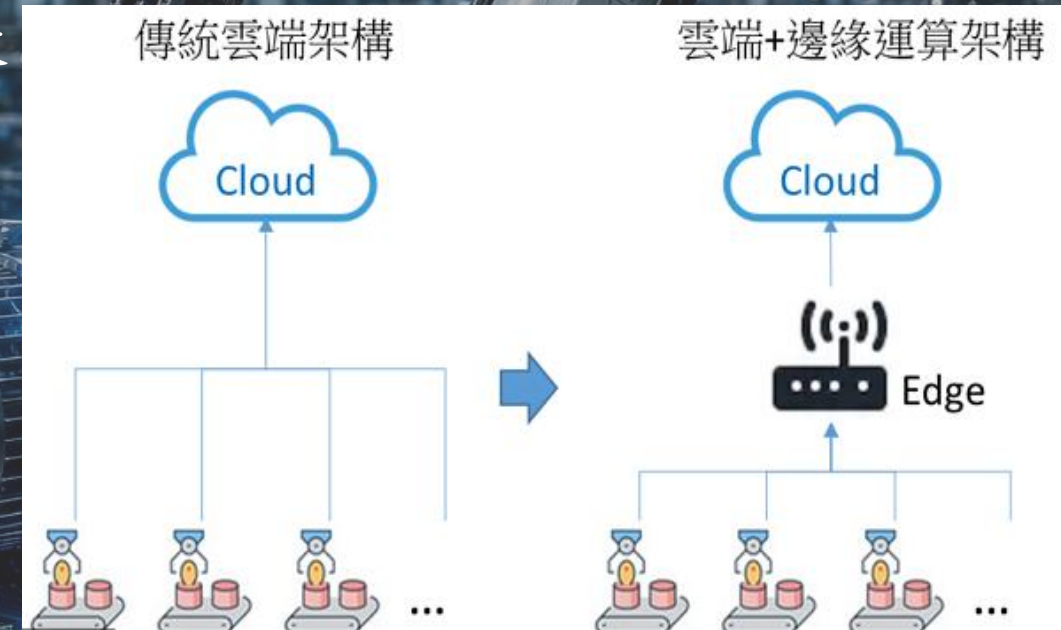
- 1. 設備互連與感知
結合 5G/6G、LPWAN、Wi-Fi 7 與 Mesh 網狀路
- 2. 運算架構與智慧
由單純雲端提升至「邊緣運算」實現低延遲響應
- 3. 人機介面與體驗
結合語音控制、手勢辨識與微互動等功能
- 4. 系統安全
物聯網必須建立硬體級安全晶片、動態金鑰與資料傳輸加密機制





邊緣運算(Edge Computing)

- 為何需要：
 - 網路邊緣的線上設備數量迅速增加，這些會產生大量數據資料(如:城市的IoT設備)，同時也消耗大量資料頻寬
- 簡介
 - 邊緣運算是一種網路運算架構，運算過程盡可能靠近資料來源以減少延遲和頻寬使用，最大限度的減少異地用戶端和伺服器之間必須發生的通信量。近年來技術的快速發展使硬體趨向小型化、高密度以及軟體的虛擬化，讓邊緣運算的應用度更加可行
 - 邊緣運算是加速互聯網的主要關鍵



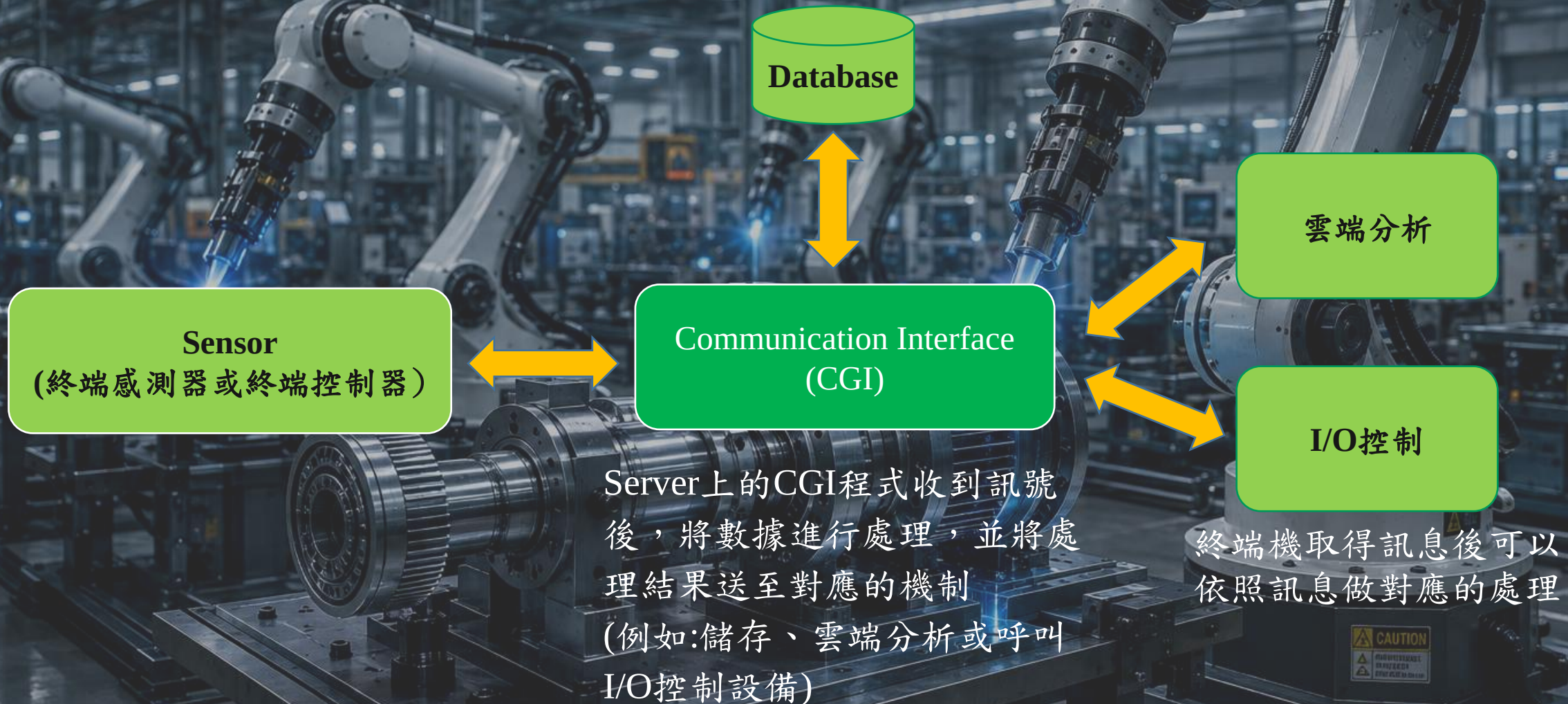


各式感測器與基本原理

- 感測器(Sensor)是由對周圍環境或者人體身上的某些物理化學反應，轉成可量化的電訊號，常見的有溫度感應器、光敏感應器、陀螺儀、加速計、影像感測器等、深度感測器……等
- 大部分的感測器本身沒有辦法直接利用，需要放大訊號以便讀取，所以通常會接到微電腦控制器上面，這讓應用上有極大好處，無論是做有線無線傳輸訊號到遠端機器上，都提高了便利性與增加應用擴充性，如下圖所示



IOT架構範例





IOT與CGI關係範例(智慧漁業為例)





CGI簡介

- 通用閘道器介面 (Common Gateway Interface, CGI)
- CGI是一種重要的網際網路技術，可以讓一個用戶端，從網頁瀏覽器向執行在網路伺服器上的程式請求資料。CGI描述了伺服器和請求處理程式之間傳輸資料的一種標準
- 早期實現CGI經常使用程式語言Perl，但CGI的一個目的是要獨立於任何語言的，Web伺服器無須在這個問題上對語言有任何了解
- CGI程式基本上可以使用任何程式實現，只要這個語言可以在這個系統上執行
- 本次CGI實作會以Node.js作示範介紹



CGI簡介

- 除Perl拿來實現CGI外，像Unix shell script、Python、Ruby、PHP、C/C++、Java、JavaScript、Node.JS等.....都可以用來編寫CGI程式





CGI簡介

- CGI常用的環境變數（僅供參考，不同的伺服器可能提供不同的環境變數）

伺服器名稱
ex.www.test.com

網頁使用port
預設為80

CGI程式所放
置的位置

Environment Variable	Description
SERVER_SOFTWARE	The type of Web server running the CGI program
SERVER_NAME	The name of the Web server host
SERVER_PORT	The port address of the Web server
GATEWAY_INTERFACE	The version number of the CGI standard
SERVER_PROTOCOL	The version of HTTP the server is running
REQUEST_METHOD	The method of requesting data specified by the client
QUERY_STRING	Request parameters supplied by the client
SCRIPT_NAME	The resource locator of the CGI program
REMOTE_HOST	The name of the client host
REMOTE_ADDR	The Ipaddress if the client host
AUTH_TYPE	Authorization method,often blank
REMOTE_USER	The name of the user provided by the client
REMOTE_IDENT	An identify fot the client user,not often available
REFERER_URL	How the client got here
HTTP_ACCEPT	The MIME types accepted by the client
HTTP_USER_AGENT	The client browser type
CONTENT_TYPE	The MIME type of data supplied with the request



CGI的用途為何?

- 依照不同身分執行不同權限的動作
 - 可以依照不同登入的身分給予不同的操作權限
- 用於IoT
 - 透過網頁控制硬體
 - 透過行動裝置監控硬體資源
 - 可以設計對應的程式控制嵌入式平台上的GPIO
 - 可以設計對應的程式控制RS-232介面
- 跨平台的操作介面
 - 主要介面為網頁，可以運行在任何有瀏覽器的裝置
 - 可以設計對應的API透過Http Request來讓其他程式呼叫使用



Node.js 簡介



- Node.js 以 JavaScript 語言為基礎，是 Ryan Dahl 基於 Google Chrome V8 引擎於 2009 年釋出的 JavaScript 開發平台，是一個高效能、易擴充的網站應用程式開發框架 (Web Application Framework)
- 目的是為了讓開發者能夠更容易開發高延展性的網路服務，不需要經過太多複雜的調校、效能調整及程式修改，就能滿足網路服務在不同發展階段對效能的要求



Node.js 安裝

- 開啟Terminal連線到TX2後輸入以下指令安裝Node.js
 - `$ sudo apt-get install nodejs`

```
nvidia@nvidia-desktop: ~/nodejs
nvidia@nvidia-desktop:~$ sudo apt install nodejs
[sudo] password for nvidia:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
 apt-clone archdetect-deb bogl-bterm busybox-static cryptsetup-bin
 dpkg-repack gir1.2-timexonmap-1.0 gir1.2-xkl-1.0 grub-common
 kde-window-manager kinit kio kpackageutils kwayland-data kwine-common
 kwine-data kwine-x11 libdebian-installer4 libkdecoration2-5v5
 libkdecoration2private5v5 libkf5activities5 libkf5attica5
 libkf5completion-data libkf5completion5 libkf5declarative-data
 libkf5declarative5 libkf5doctools5 libkf5globalaccel-data libkf5globalaccel5
 libkf5globalaccelprivate5 libkf5idle5 libkf5i18n5 libkf5i18n-data5
```



Node.js 安裝 (1/2)

- 官方也有提供各種版本的安裝包供使用者下載
- <https://nodejs.org/zh-tw/download/>



The screenshot shows the Node.js download page. At the top is the Node.js logo and a navigation bar with links: 首頁, 關於我們, 下載, 文件, 加入我們, 安全, 相關認證, 部落格. Below the navigation bar is a section titled '下載' (Download) with the text '最新版: 14.16.1 (包含 npm 6.14.12)'. It says '下載適合您平台的 Node.js 原始碼或安裝套件。立刻開始使用 Node.js。' (Download Node.js source code or installation packages suitable for your platform. Start using Node.js immediately.). There are two main columns: 'LTS' (Long Term Support) with the text '建議大部分使用者使用' (Recommended for most users) and '目前版本' (Current version) with the text '最新功能' (Latest features). Under the LTS column are three options: 'Windows 安裝包' (Windows installer) with the Windows logo and file name 'node-v14.16.1-x64.msi', 'macOS 安裝包' (macOS installer) with the Apple logo and file name 'node-v14.16.1.pkg', and '原始碼' (Source code) with a cube icon and file name 'node-v14.16.1.tar.gz'.

node

首頁 | 關於我們 | 下載 | 文件 | 加入我們 | 安全 | 相關認證 | 部落格

下載

最新版: 14.16.1 (包含 npm 6.14.12)

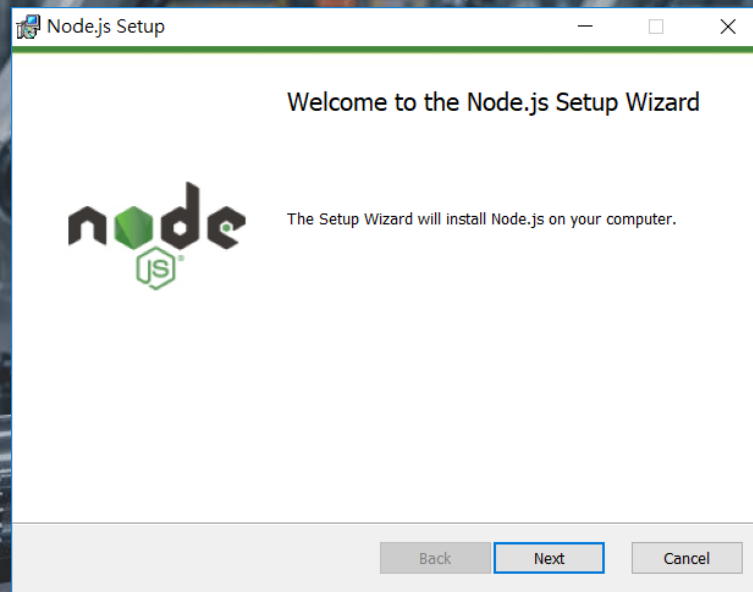
下載適合您平台的 Node.js 原始碼或安裝套件。立刻開始使用 Node.js。

LTS 建議大部分使用者使用	目前版本 最新功能	
 Windows 安裝包 node-v14.16.1-x64.msi	 macOS 安裝包 node-v14.16.1.pkg	 原始碼 node-v14.16.1.tar.gz



Node.js 安裝 (2/2)

- Next & Next ...





npm簡介

- npm(全稱Node Package Manager)是Node.js預設的、用JavaScript編寫的軟體套件管理系統。
- npm目前累計共有數十萬的開源模組來讓開發人員下載，其中有許多跨國企業將npm作為開發工具。





npm模組

- npm包含多種模組，可以到以下網址搜尋適合的模組安裝到電腦裡
- <https://www.npmjs.com/package/package>
- 這裡以express為例

express 

4.17.1 • Public • Published 2 years ago

 Readme  Explore  30 Dependencies 52,534 Dependents 264 Versions

express

Fast, unopinionated, minimalist web framework for **node**.

npm v4.17.1 downloads 71M/month linux passing windows passing coverage 100%

```
const express = require('express')
const app = express()

app.get('/', function (req, res) {
  res.send('Hello World')
})

app.listen(3000)
```

Install

```
> npm i express
```

Weekly Downloads

13,668,979

Version	License
4.17.1	MIT
Unpacked Size	Total Files
208 kB	16
Issues	Pull Requests
108	60

Homepage

 expressjs.com/

下載此套件的指令

每周使用者下載次數



npm安裝

- 接續剛剛安裝完的Node.js，一樣在TX2的Terminal上輸入以下指令
- `$ sudo apt install npm`

```
nvidia@nvidia-desktop: ~/nodejs
nvidia@nvidia-desktop:~/nodejs$ sudo apt install npm
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
 apt-clone archdetect-deb bogl-bterm busybox-static cryptsetup-bin
 dpkg-repack gir1.2-timezonemap-1.0 gir1.2-xkl-1.0 grub-common
 kde-window-manager kinit kio kpackage5 kwayland-data kwin-common
 kwin-data kwin-x11 libdebian-installer4 libkdecorations2-5v5
 libkdecorations2private5v5 libkf5activities5 libkf5attica5
 libkf5completion-data libkf5completion5 libkf5declarative-data
```



確認Nodejs及npm安裝完成及版本資訊

- 安裝完nodejs及npm後，可輸入以下指令，確認完成安裝及版本資訊
 - \$ nodejs --version
 - \$ npm --version

```
nvidia@ubuntu:~/Desktop$ nodejs --version  
v10.19.0  
nvidia@ubuntu:~/Desktop$ npm --version  
6.14.4
```



npm 專案初始化

- 安裝完後進到指定的資料夾(範例資料夾名稱為nodejs)，進行初始化，並且產生package.json檔案
- \$ npm init

```
nvidia@nvidia-desktop:~/nodejs$ npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help json` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg> --save` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
name: (nodejs)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to /home/nvidia/nodejs/package.json:

{
  "name": "nodejs",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}

Is this ok? (yes)
nvidia@nvidia-desktop:~/nodejs$
```



專案內的package.json

- package.json檔案：以下為package.json格式，使用npm init生成時，輸入 yes 可自動從當前目錄提取資訊生成預設值，也可由同學們自行定義專案版本等資訊

當前模組名稱

當前模組版本初
始預設為1.0.0

指定了專案載入的入口檔案

```
Press ^C at any time to quit.  
name: (nodejs)  
version: (1.0.0)  
description:  
entry point: (index.js)  
test command:  
git repository:  
keywords:  
author:  
license: (ISC)
```



express模組介紹

- express 能快速架設web server，是npm社群上受歡迎的Node web 框架函式庫，該模組提供以下功能：
 - 替不同HTTP Method、不同URL路徑的requests編寫不同的處理方法
 - 透過整合「畫面」的渲染引擎來達到插入資料到樣板中產生response
 - 常見的web應用設定，例：連線用的port和產生response的樣板位置
 - 在request的處理流程中增加額外的「中間層」進行處理

express



使用npm安裝express模組

- 安裝模組
 - `$ npm i express --save`
- i表示install，也可將此行指令換成 `$ npm install express --save`
- `--save` 是指該模組資訊會記錄在package.json檔裡

```
nvidia@nvidia-desktop:~/nodejs$ npm i express --save
nodejs@1.0.0 /home/nvidia/nodejs
├── express@4.17.3
│   ├── accepts@1.3.8
│   │   ├── mime-types@2.1.35
│   │   │   ├── mime-db@1.52.0
│   │   │   └── negotiator@0.6.3
│   ├── array-flatten@1.1.1
│   ├── media-typer@0.3.0
│   ├── utils-merge@1.0.1
│   └── vary@1.1.2
└── nodejs@1.0.0 No description
npm WARN nodejs@1.0.0 No repository field.
nvidia@nvidia-desktop:~/nodejs$
```

安裝express模組可能會碰到的問題





無法安裝問題

通常是TX2沒連到網路

```
nvidia@nvidia-desktop:~/Lab04$ npm i express --save
npm error code ENOTFOUND
npm error syscall getaddrinfo
npm error errno ENOTFOUND
npm error network request to https://registry.npmjs.org/express failed, reason: getaddrinfo ENOTFOUND registry.npmjs.org
npm error network This is a problem related to network connectivity.
npm error network In most cases you are behind a proxy or have bad network settings.
npm error network
npm error network If you are behind a proxy, please make sure that the
npm error network 'proxy' config is set properly. See: 'npm help config'

npm error A complete log of this run can be found in: /home/nvidia/.npm/_logs/2025-10-22T09_24_25_635Z-debug-0.log
```



解決方法——連線WiFi

- nmcli device wifi list
查看wifi有哪些可以連線的

```
nvidia@nvidia-desktop:~/Lab04$ nmcli device wifi list
```

IN-USE	SSID	MODE	CHAN	RATE	SIGNAL	BARS	SECURITY
	NTUT	Infra	64	270 Mbit/s	74		--
	NTUT-Guest	Infra	64	270 Mbit/s	74		--
	NTUT-802.1X	Infra	64	270 Mbit/s	72		WPA2 802.1X
	TANetRoaming	Infra	64	270 Mbit/s	72		--
	xx0o#\$@_@\$#o0Xx	Infra	11	130 Mbit/s	64		WPA2
	NTUT	Infra	1	130 Mbit/s	57		--
	NTUT-Guest	Infra	1	130 Mbit/s	57		--
	TANetRoaming	Infra	1	130 Mbit/s	57		--
	NTUT-802.1X	Infra	1	130 Mbit/s	57		WPA2 802.1X
	Tai 的 S24	Infra	6	130 Mbit/s	54		WPA2
	LAPTOP-14T1JR5L 4804	Infra	11	130 Mbit/s	49		WPA2
	Nokia G50	Infra	6	117 Mbit/s	44		WPA2
	6G	Infra	6	270 Mbit/s	29		WPA2
	NTUT-Guest	Infra	6	130 Mbit/s	25		--
	for_review	Infra	11	130 Mbit/s	22		WPA2
	ACLab-DlinkNew	Infra	9	270 Mbit/s	19		WPA2



解決方法——選擇網路連線

`sudo nmcli device wifi connect 'Wifi名稱' password "Wifi密碼"`

```
nvidia@nvidia-desktop:~/Lab04$ sudo nmcli device wifi connect '網路名稱' password "網路密碼"
Device 'wlan0' successfully activated with '1ed17c4f-f109-4b7f-a81c-67eb685f3025'.
```

`ping -c 3 8.8.8.8`

有回應代表成功連上網路

```
nvidia@nvidia-desktop:~/Lab04$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=121 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=64.2 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=61.2 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 61.286/82.322/121.463/27.704 ms
```



解決方法——檢查DNS

getent hosts registry.npmjs.org
若沒有回應代表DNS有問題

```
nvidia@nvidia-desktop:~/Lab04$ getent hosts registry.npmjs.org
2606:4700::6810:323 registry.npmjs.org
2606:4700::6810:123 registry.npmjs.org
2606:4700::6810:1b22 registry.npmjs.org
2606:4700::6810:1f22 registry.npmjs.org
2606:4700::6810:23 registry.npmjs.org
2606:4700::6810:1e22 registry.npmjs.org
2606:4700::6810:1822 registry.npmjs.org
2606:4700::6810:1922 registry.npmjs.org
2606:4700::6810:1c22 registry.npmjs.org
2606:4700::6810:1d22 registry.npmjs.org
2606:4700::6810:223 registry.npmjs.org
2606:4700::6810:1a22 registry.npmjs.org
```

cat /etc/resolv.conf
通常指向 127.0.0.53

```
nvidia@nvidia-desktop:~/Lab04$ cat /etc/resolv.conf
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
# 127.0.0.53 is the systemd-resolved stub resolver.
# run "systemd-resolve --status" to see details about the actual nameservers.

nameserver 127.0.0.53
```

檢查完都沒問題後再次嘗試 \$ npm i express --save



Nodejs 引入模組-require()

- // Importing a local module
- // working directory(On Windows, this would resolve to
.\path\myLocalModule.)
- `const myLocalModule = require('./path/myLocalModule');` local module
- // Importing a JSON file:
- `const jsonData = require('./path/filename.json');` JSON File
- // Importing a module from node_modules or Node.js built-in module:
- `const child_process = require('child_process');` npm module Node.js built-in module



啟動網頁伺服器

- 建立index.js檔，利用express套件快速架設web server並在前端顯示Hello World!!)

index.js

```
1 //include necessary npm package
2 const express = require("express");
3 const app = express();
4
5 //simple routes
6 app.get("/", (req,res)=>{
7   res.send("Hello World!!")
8 })
9
10 // set port, listen for requests
11 const PORT = process.env.PORT || 8080;
12 app.listen(PORT, () => {
13   console.log(`Server is running on port ${PORT}.`);
14 });
```

引入npm模組

建立GET/路由
回傳Hello World

伺服器指定port



啟動網頁伺服器

- 程式碼撰寫完後，開啟終端機，切換到專案目錄
- 輸入 `node ./index.js` 即可啟動網頁伺服器

```
nvidia@nvidia-desktop:~/nodejs$ node ./index.js  
Server is running on port 8080.
```



在遠端查看網頁

- 在遠端使用瀏覽器連進嵌入式平台伺服器
- (網址為TX2嵌入式平台的ip及port)

TX2的網路IP:8080

← → ↻ ⚠ 不安全 | 192.168.137.69:8080

Hello World!!



簡易 web server 範例-客戶端傳送參數

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=\\, initial-scale=1.0">
6   <title>Nodejs</title>
7 </head>
8 <body>
9   <p>
10     <h1>Nodejs</h1>
11   </p>
12   <hr>
13   <form action="/index" method="get">
14     <h2>SEND REQUEST</h2>
15     <label>姓名<input type="text" name="name"></label>
16     <br>
17     <label>學號<input type="text" name="id"></label>
18     <hr>
19     <input type="submit" value="Submit">
20   </form>
21 </body>
22 </html>
```

index.html

將表單以GET的方式傳送/index 路由

姓名的參數為name

學號的參數為id



簡易 web server 範例-伺服器端接收參數

```
1  //include necessary npm package
2  const express = require("express");
3  const app = express();
4
5  //static resources
6  app.use(express.static('./public'));
7
8  //simple routes
9  app.get("/index", (req,res)=>{
10     var response = {
11         "my name": req.query.name,
12         "my id": req.query.id
13     }
14     res.send(response)
15 })
16
17 // set port, listen for requests
18 const PORT = process.env.PORT || 8080;
19 app.listen(PORT, () => {
20     console.log(`Server is running on port ${PORT}.`);
21 });
```

index.js

指定靜態檔案的存放路徑

建立簡易路由:

此例是以GET接收/index路由的參數
req.query.name 接收客戶端傳來name
req.query.id 接收客戶端傳來id
res.send()則是將資訊回傳至客戶端



簡易 web server 範例-專案架構

專案資料夾

✓ NODEJS

npm modules

> node_modules

指定靜態網頁路徑

✓ public

✎ index.html

專案 Entry Point

JS index.js

{ } package.json



網頁顯示畫面

範例輸入:
姓名:betty
學號:110598004

Nodejs

SEND REQUEST

姓名
學號

Submit

Send Request

localhost:8080/index?name=Lucas&id=109598999

`{"my name": "Lucas", "my id": "109598999"}`

回應內容



Html - table

```
<!DOCTYPE html>
<html>
<head>

</head>
<body>

<h2>Cell that spans two rows:</h2>
<table border="1">
  <tr>
    <th>Name:</th>
    <td>Bill Gates</td>
  </tr>
  <tr>
    <th rowspan="2">Telephone:</th>
    <td>55577854</td>
  </tr>
  <tr>
    <td>55577855</td>
  </tr>
</table>

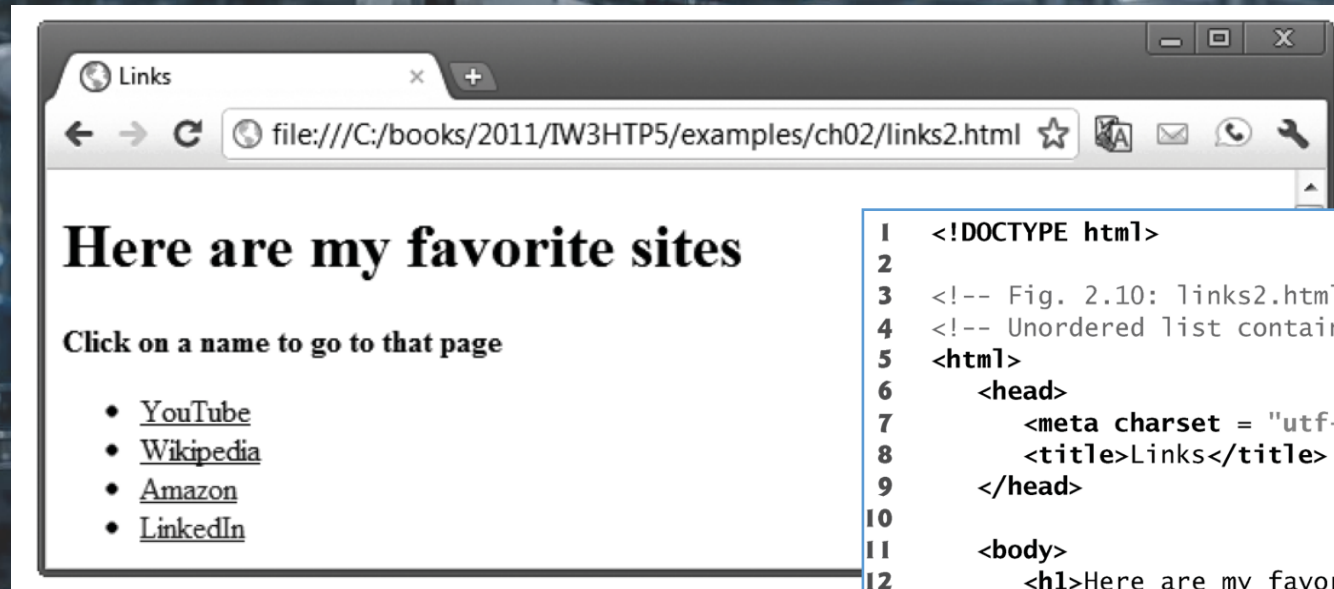
</body>
</html>
```

Cell that spans two rows:

Name:	Bill Gates
Telephone:	55577854
	55577855



Html - list



```
1  <!DOCTYPE html>
2
3  <!-- Fig. 2.10: links2.html -->
4  <!-- Unordered list containing hyperlinks. -->
5  <html>
6    <head>
7      <meta charset = "utf-8">
8      <title>Links</title>
9    </head>
10
11   <body>
12     <h1>Here are my favorite sites</h1>
13     <p><strong>Click on a name to go to that page</strong></p>
14
15     <!-- create an unordered list -->
16     <ul>
17       <!-- the list contains four list items -->
18       <li><a href = "http://www.youtube.com">YouTube</a></li>
19       <li><a href = "http://www.wikipedia.org">Wikipedia</a></li>
20       <li><a href = "http://www.amazon.com">Amazon</a></li>
21       <li><a href = "http://www.linkedin.com">LinkedIn</a></li>
22     </ul>
23   </body>
24 </html>
```



Html – radio

```
<fieldset>
  <legend>Select a maintenance drone:</legend>

  <div>
    <input type="radio" id="huey" name="drone" value="huey"
      checked>
    <label for="huey">Huey</label>
  </div>

  <div>
    <input type="radio" id="dewey" name="drone" value="dewey">
    <label for="dewey">Dewey</label>
  </div>

  <div>
    <input type="radio" id="louie" name="drone" value="louie">
    <label for="louie">Louie</label>
  </div>
</fieldset>
```

Select a maintenance drone:

- ☒ Huey
- ☐ Dewey
- ☐ Louie



Html – checkbox

```
<fieldset>
  <legend>Choose your monster's features:</legend>

  <div>
    <input type="checkbox" id="scales" name="scales" checked>
    <label for="scales">Scales</label>
  </div>

  <div>
    <input type="checkbox" id="horns" name="horns">
    <label for="horns">Horns</label>
  </div>
</fieldset>
```

Choose your monster's features:

- ☒ Scales
- ☐ Horns



Html – form1

```
<!DOCTYPE html>
<html>
<body>
```

```
<form action="action_page.php">
  First name:<br>
  <input type="text" name="firstname" value="Mickey">
  <br>
  Last name:<br>
  <input type="text" name="lastname" value="Mouse">
  <br><br>
  <input type="submit" value="Submit">
</form>
```

<p>If you click the "Submit" button, the form-data will be called "action_page.php".</p>

```
</body>
</html>
```

First name:

Mickey

Last name:

Mouse

Submit



Html – form2

Forms

file:///C:/books/2011/TW3HTP5/examples/ch02/form.html

Feedback Form

Please fill out this form to help us improve our site.

Name:

```
1 <!DOCTYPE html>
2
3 <!-- Fig. 2.14: form.html -->
4 <!-- Form with a text field and hidden fields. -->
5 <html>
6   <head>
7     <meta charset = "utf-8">
8     <title>Forms</title>
9   </head>
10
11   <body>
12     <h1>Feedback Form</h1>
13
14     <p>Please fill out this form to help
15       us improve our site.</p>
16
17     <!-- this tag starts the the form, gives the -->
18     <!-- method of sending information and the -->
19     <!-- location of the form-processing script -->
20     <form method = "post" action = "http://www.deitel.com">
21       <!-- hidden inputs contain non-visual -->
22       <!-- information that will also be submitted -->
23       <input type = "hidden" name = "recipient"
24         value = "deitel@deitel.com">
```

```
25     <input type = "hidden" name = "subject"
26       value = "Feedback Form">
27     <input type = "hidden" name = "redirect"
28       value = "main.html">
29
30     <!-- <input type = "text"> inserts a text field -->
31     <p><label>Name:
32       <input name = "name" type = "text" size = "25"
33         maxlength = "30">
34     </label></p>
35
36     <p>
37       <!-- input types "submit" and "reset" insert -->
38       <!-- buttons for submitting and clearing the -->
39       <!-- form's contents, respectively -->
40       <input type = "submit" value = "Submit">
41       <input type = "reset" value = "Clear">
42     </p>
43   </form>
44 </body>
45 </html>
```



補充(Links2 Web 瀏覽器)

- Links是Twibright實驗室編寫的web瀏覽器，而Links2是基於它的一個圖形化版本。Links2支持滑鼠點擊，設計強調速度，不支持任何CSS，在一定程度上很好地支持了HTML和JavaScript。在以上實作中可利用此套件在terminal開啟網頁。
- 透過以下指令瀏覽Internet：
 - Command: links2 [URL]



補充(Links2 Web 瀏覽器)

```
nvidia@nvidia-desktop:~/nodejs$ sudo apt-get install links2
```

```
nvidia@nvidia-desktop:~/nodejs$ links2 192.168.137.69:8080
```

```
Hello World!!
```



補充(Links2 Web 瀏覽器)

Nodejs

SEND REQUEST

name 12132_____

ntut_id 2123_____

[Submit]

+----- Unknown type -----+

| Content type is application/json. |

| Do you want to save or display this file? |

| [Save] [Display] [Cancel] |

+-----+

→ {"my name": "12132", "my id": "2123"}